

AU426

Classical feedback control with for an inverted pendulum and a 2DOF helicopter

Cong Khanh DINH

2025

The aim of this lab session is to design, synthesize, and implement fundamental control algorithms introduced in the course, using two experimental platforms: the QUBE-Servo, a compact rotary servo system (SIMO), and the Quanser Aero 2, a 2-DOF helicopter system (MIMO). In this practical session, Proportional-Derivative(PD) and Linear Quadratic Regulator(LQR) controllers will be developed. The design process begins with mathematical modeling of each system, followed by validation through a digital twin environment implemented in MATLAB and Simulink. This approach ensures that controller performance can be verified in simulation before deployment on the physical hardware. Finally, the designed controllers are implemented and tested on the real systems using the QUARC real-time control software, allowing a direct comparison between simulated and experimental results.

Contents

1	Introduction	3
1.1	Qube-Servo 2	3
1.1.1	Interaction between the different hardware components	4
1.1.2	Model of the Qube-Servo inverted pendulum	4
1.1.3	Linear model	7
1.2	Aero 2	7
1.2.1	Model of Aero 2-DOF helicopter	8
1.2.2	Linear model	10
2	Exercises	11
2.1	State-space modeling	11
2.1.1	Rotary pendulum	11
2.1.2	2-DOF helicopter	13
2.2	Design and implement of PD controller using pole placement method . . .	14
2.2.1	Rotary pendulum	15
2.2.2	2-DOF helicopter	16
2.3	Design and implement LQR controller	17
2.3.1	Rotary pendulum	18
2.3.2	2-DOF helicopter	19

1 Introduction

In this lab, the focus is placed on two experimental setups: the rotary inverted pendulum and the Aero 2 two-degree-of-freedom (2-DOF) helicopter. These platforms provide practical examples of challenging control problems and allow the application of modern control techniques studied in the course.

The DC servo motor system underlying the QUBE-Servo is an important introductory platform. In this session, we provide only a brief overview of DC motor fundamentals; for a more detailed study, students are referred to TP1 of the PAU101 course. Instead, the emphasis of this lab is placed on more advanced systems with multiple inputs and outputs, which illustrate key issues in nonlinear dynamics, underactuation, and multi-variable control.

The experiments covered in this lab are:

1. Feedback control of the rotary inverted pendulum.
2. Feedback control of the 2-DOF helicopter (Aero 2).

Both setups are interfaced with MATLAB/Simulink, which serves as the main environment for modeling, simulation, and real-time controller implementation. This integration of software and hardware is supported by the Quanser Interactive Labs, providing a digital twin framework. Through the QUARC software in Simulink, students can seamlessly switch between the digital twin and the physical system, ensuring that controller designs are first validated in simulation before being deployed on the actual hardware.

1.1 Qube-Servo 2

The Quanser QUBE-Servo 2, shown in Figure 1, is a small rotary servo platform designed for a wide range of classical servo control and inverted pendulum experiments. The unit can be connected to a computer through a USB interface for control. The system is driven using a direct-drive 18V brushed DC motor. The motor is powered by a built-in Pulse Width Modulation (PWM) amplifier with integrated current sense. Two add-on modules are supplied with the system:

- an inertia disc
- a rotary pendulum.

The two modules can be easily attached or interchanged using magnets mounted on the QUBEServo 3 module connector. Single-ended rotary encoders are used to measure the angular position of the DC motor and pendulum, while the angular velocity of the motor can be either estimated from the angular position encoder-based measurement or directly measured using an integrated software-based tachometer.



Figure 1: QUBE-Servo 2 with 2 modules: a) QUBE-Servo 2 with inertial disc module; b) QUBE-Servo 2 with rotary inverted pendulum module.

1.1.1 Interaction between the different hardware components

The interaction between the different system components on the QUBE-Servo system is illustrated in Figure 2. On the data acquisition (DAQ) device block, the motor and pendulum encoders are connected to the Encoder Input (EI) channels #0 and #1. The Analog Output (AO) channel is connected to the power amplifier command, which then drives the DC motor.

The DAQ Analog Input (AI) channel is connected to the PWM amplifier current sense circuitry. The DAQ also controls the integrated tri-colour LEDs via an internal serial data bus. The DAQ can be interfaced to the PC or laptop via USB link in the QFLEX 2 USB, or to an external microcontroller via SPI in the QFLEX 2 Embedded.

1.1.2 Model of the Qube-Servo inverted pendulum

The DC motor shaft is connected to the load hub(See Figure 3). The hub is a metal disk used to mount the disk or rotary pendulum and has a moment of inertia of J_h . A load(either inertial disc or pendulum) is attached to the output shaft with a moment of inertia of J_d . The back-emf (electromotive) voltage $e_b(t)$ depends on the speed of the motor shaft, ω_m , and the back-emf constant k_m of the motor, km. It opposes the current flow. The back-emf voltage is given by:

$$e_b(t) = k_m \omega_m(t) \quad (1)$$

Using Kirchoff's Voltage Law, we can write the following equation:

$$v_m(t) - R_m i_m(t) - L_m \frac{di_m(t)}{dt} - k_m \omega_m(t) = 0. \quad (2)$$

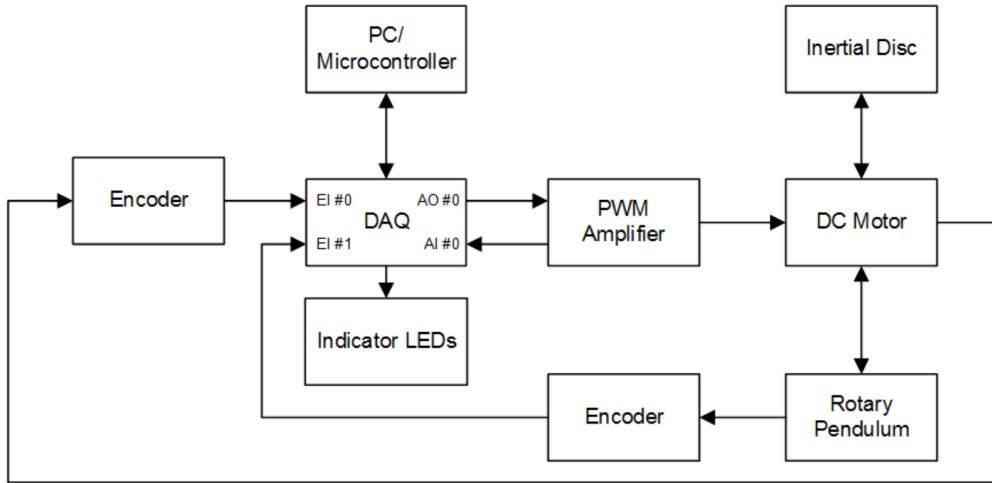


Figure 2: Interaction between components in Qube-Servo system

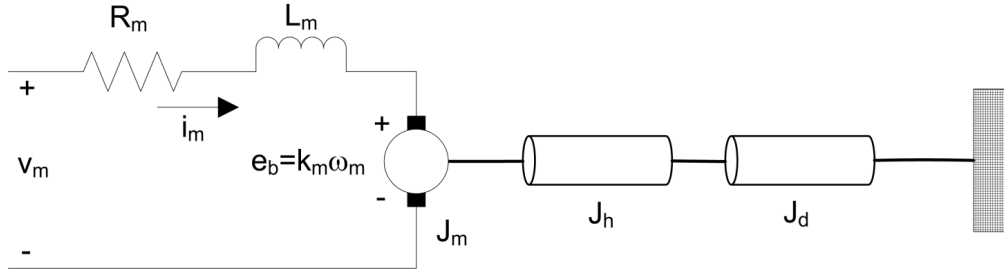


Figure 3: QUBE-Servo DC motor and load

Since the motor inductance L_m is much less than its resistance, it can be ignored. Then, the equation becomes

$$v_m(t) - R_m i_m(t) - k_m \omega_m(t) = 0. \quad (3)$$

Solving for $i_m(t)$, the motor current can be found as:

$$i_m(t) = \frac{v_m(t) - k_m \omega_m(t)}{R_m}. \quad (4)$$

The motor shaft equation is expressed as:

$$J_{eq} \dot{\omega}_m(t) = \tau_m(t), \quad (5)$$

where J_{eq} is total moment of inertia acting on the motor shaft and τ_m is the applied torque from the DC motor. Based on the current applied, the torque is

$$\tau_m = k_t i_m(t), \quad (6)$$

The rotary inverted pendulum system has been widely studied and as a consequence the physics-informed model has been established and is available.

As the model of the inverted and non-inverted pendulum are very close (they only differ with some minus signs for some coefficients) we will first deal with the non-inverted pendulum. The rotary pendulum model is shown in Figure 4.

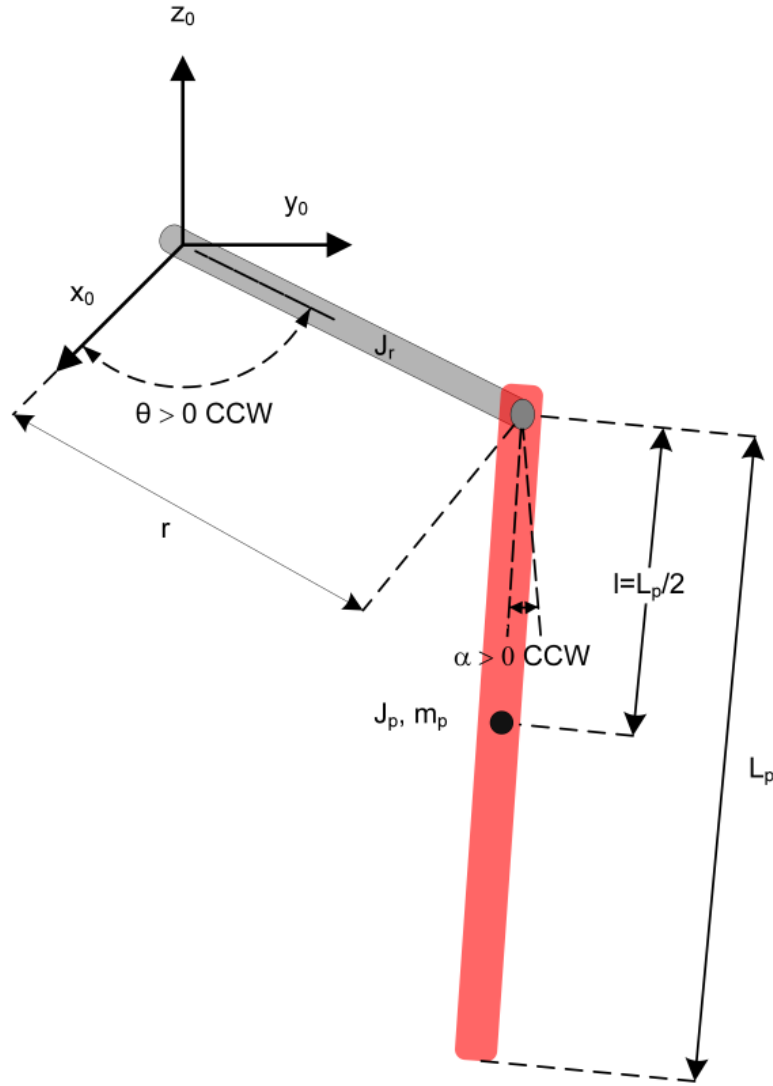


Figure 4: Rotary pendulum model(Non-inverted model)

The rotary arm pivot is attached to the QUBE-Servo system and is actuated. The arm has a length of r , a moment of inertia of J_r , and its angle θ increases positively when it rotates counter-clockwise (CCW). The servo (and thus the arm) should turn in the CCW direction when the control voltage is positive, $v_m > 0$.

The pendulum link is connected to the end of the rotary arm. It has a total length of

L_p and its center of mass is at $l = L_p/2$. The moment of inertia about its center of mass is J_p . The rotary pendulum angle α is zero when it is hanging downward and increases positively when rotated CCW.

The equations of motion (EOM) for the pendulum system were developed using the Euler-Lagrange method. This systematic method is often used to model complicated systems such as robot manipulators with multiple joints. The total kinetic and potential energy of the system is obtained, then the Lagrangian can be found.

The resultant nonlinear EOM are:

$$J_\theta \ddot{\theta} + 2m_p l^2 \sin \alpha \cos \alpha \dot{\theta} \dot{\alpha} + m_p r l (\cos \alpha \ddot{\alpha} - \sin \alpha \dot{\alpha}^2) = \tau_m - b_r \dot{\theta} \quad (7)$$

$$J_\alpha \ddot{\alpha} - m_p l^2 \sin \alpha \cos \alpha \dot{\theta}^2 + m_p r l \cos \alpha \ddot{\theta} + m_p g l \sin \alpha = -b_p \dot{\alpha} \quad (8)$$

where $J_\theta = J_r + m_p r^2 + m_p l^2 \sin^2 \alpha$ and $J_\alpha = J_p + m_p l^2$. The viscous damping acting on the rotary arm and the pendulum link are b_r and b_p , respectively.

*Question**. Derive the EOM of the rotary pendulum. The results should look like (7) and (8).

The state and output are defined:

$$x(t) = [\theta(t) \quad \alpha(t) \quad \dot{\theta}(t) \quad \dot{\alpha}(t)]^\top \quad (9)$$

$$y(t) = [\theta(t) \quad \alpha(t)]^\top \quad (10)$$

The system state x defines the state variables needed to model the system and output state y defines the state variables that are measured directly.

1.1.3 Linear model

When the nonlinear model above is linearized about the operating point, e.g. downward position ($\alpha = 0$) and $\Theta(\dot{\theta}^2, \dot{\alpha}^2, \dot{\theta}^2 \dot{\alpha}) \ll$ negligible, the resultant linear EOM for the rotary pendulum are defined as

$$J_\theta \ddot{\theta} + m_p r l \ddot{\alpha} \operatorname{sgn}(\cos \bar{\alpha}) = \tau_m - b_r \dot{\theta} \quad (11)$$

$$J_\alpha \ddot{\alpha} + m_p r l \ddot{\theta} \operatorname{sgn}(\cos \bar{\alpha}) + m_p g l \operatorname{sgn}(\cos \bar{\alpha}) \alpha = -b_p \dot{\alpha} \quad (12)$$

1.2 Aero 2

The Quanser Aero, pictured in Figure 5, is a compact dual-rotor two-degree-of-freedom(2-DOF) aerospace system that can be used to perform a variety of actuator and flight control based experiments.

For all versions, the system is driven using two direct-drive 18V brushed DC motors. The motors are powered by a built-in PWM amplifier with built-in current sense. Single-ended rotary encoders are used to measure the angular position of the DC motors, and the speed of the motors can be measured with a software tachometer(see Figure 5).

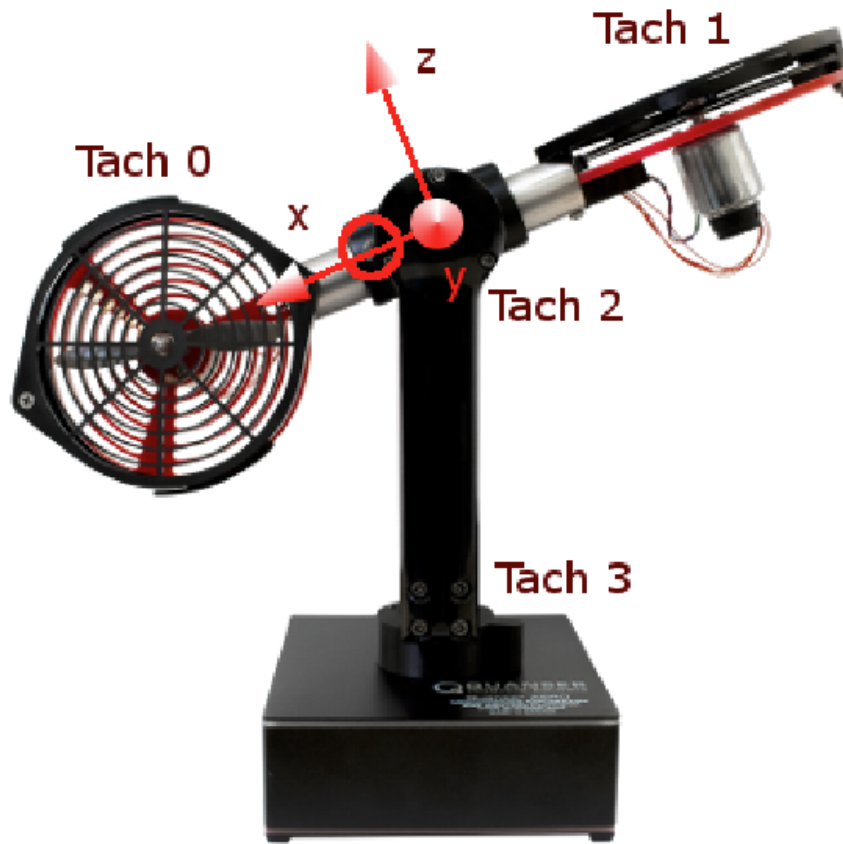


Figure 5: Aero platform with encoders and tachometers for measurement.

1.2.1 Model of Aero 2-DOF helicopter

The free-body diagram of the Quanser Aero Experiment is illustrated in Figure 6. The following conventions are used for the modeling:

1. The helicopter is horizontal and parallel with the ground when the pitch angle is zero, i.e. $\theta = 0$.
2. The pitch angle increases positively, $\dot{\theta}(t) > 0$, when the front rotor is moved upwards and the body rotates counter-clockwise (CCW) about the Y axis.
3. The yaw angle increases positively, $\dot{\psi}(t) > 0$, when the body rotates counter-clockwise (CCW) about the Z axis.
4. Pitch increases, $\dot{\theta} > 0$, when the front rotor voltage is positive $V_p > 0$.

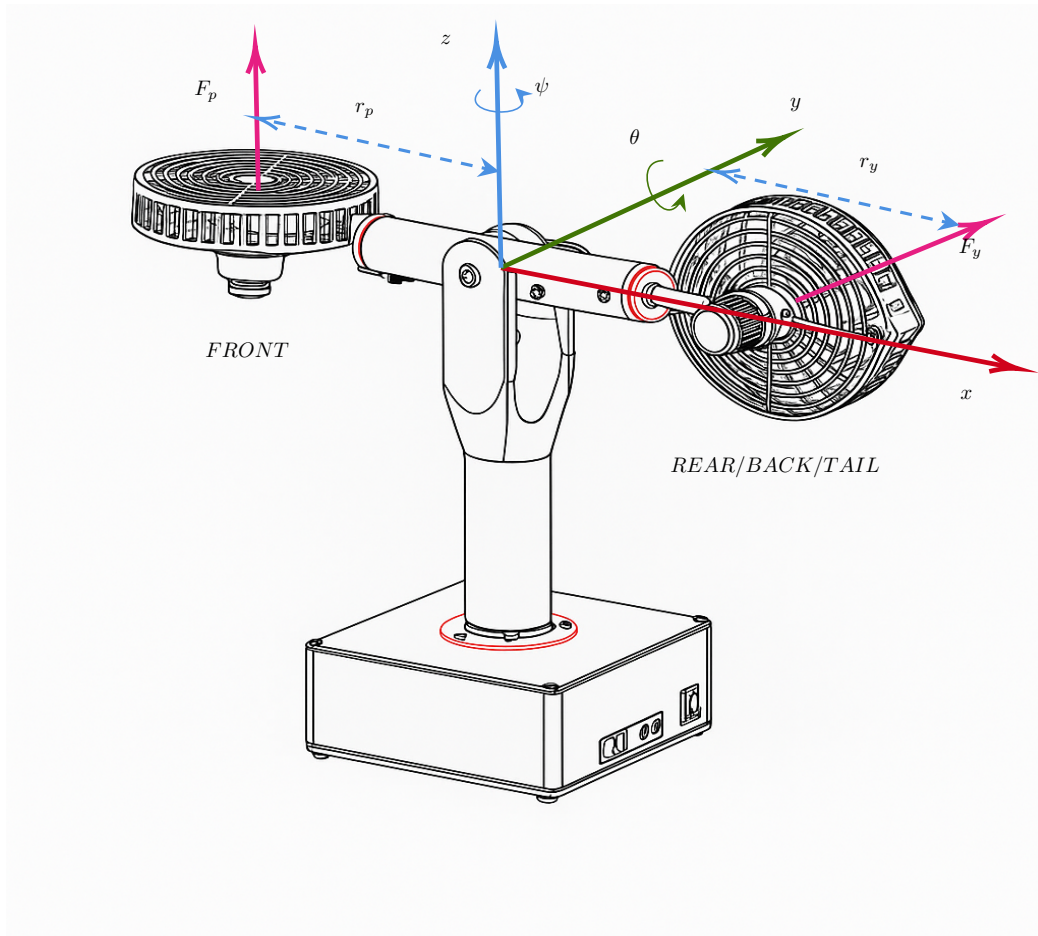


Figure 6: Simple free-body diagram of Quanser Aero Experiment.

5. Yaw increases, $\dot{\psi} > 0$, when the back (or tail) rotor voltage is positive, $V_y > 0$.

When voltage is applied to the pitch motor, V_p , the speed of rotation results in a force, F_p , that acts normal to the body at a distance r_p from the pitch axis. The rotation of the propeller generates a torque about the pitch rotor motor shaft which is in turn seen about the yaw axis. Thus rotating the pitch propeller does not only cause motion about the pitch axis but also about the yaw axis. As described earlier, that's why conventional helicopters include a tail, or anti-torque, rotor to compensate for the torque generated about the yaw axis by the large, main rotor.

Similarly, the yaw motor causes a force F_y that acts on the body at a distance r_y from the yaw axis as well as a torque about the pitch axis.

We can develop a simple linear model that takes this coupling into account, and represents the motions of the Quanser Aero about the horizontal, i.e. when the body is parallel with the ground.

1.2.2 Linear model

The equations of motion are:

$$J_p \ddot{\theta} + D_p \dot{\theta} + K_{sp} \theta = \tau_p \quad (13)$$

$$J_y \ddot{\psi} + D_y \dot{\psi} = \tau_y \quad (14)$$

where the torques acting on the pitch and yaw axes are

$$\tau_p = K_{pp} r_p V_p + K_{py} r_y V_y \quad (15)$$

$$\tau_y = K_{yp} r_p V_p + K_{yy} r_y V_y \quad (16)$$

The parameters used in the EOMs above are:

- J_p is the total moment of inertia about the pitch axis,
- D_p is the damping about the pitch axis,
- K_{sp} is the stiffness about the pitch axis,
- K_{pp} is torque-thrust gain from the pitch rotor,
- K_{py} is the cross-torque thrust gain acting on the pitch from the yaw rotor,
- V_p is the voltage applied to the pitch rotor, and
- V_y is the voltage applied to the yaw rotor motor.

Similarly, the total moment of inertia and damping about the yaw axis is J_y and D_y , respectively, K_{yy} is the torque-thrust gain from the yaw rotor, and K_{yp} is the cross torque-thrust gain acting on the yaw axis from the pitch rotor.

The total moment of inertia acting about the pitch and yaw axes are

$$J_p = J_{body} + J_{prop_f/Y} + J_{prop_r/Y} \quad (17)$$

$$J_y = J_{body} + J_{prop_f/Z} + J_{prop_r/Z} + J_{york} \quad (18)$$

where J_{body} is the moment of inertia of helicopter body about its center, $J_{prop_f/Y} = J_{prop_r/Z}$ and $J_{prop_f/Z} = J_{prop_r/Y}$ are moment of inertia of front(f) and rear(r) motors w.r.t. Y and Z axes. In practice, those motors can be considered single-point mass, which lead to equally inertia of the two propellers $J_{prop} = m_{prop} r_{prop}^2$. Finally, J_{york} is the moment of inertia for the yoke around the Z axis.

We define the state and output for the Quanser Aero:

$$x(t) = [\theta(t) \quad \psi(t) \quad \dot{\theta}(t) \quad \dot{\psi}(t)]^\top \quad (19)$$

$$y(t) = [\theta(t) \quad \psi(t)]^\top \quad (20)$$

and the control variables:

$$u = [V_p(t) \quad V_y(t)]^\top \quad (21)$$

2 Exercises

2.1 State-space modeling

The linearized state-space model of the for both systems defined above is:

$$\begin{aligned}\dot{x}(t) &= A x(t) + B u(t), \\ y(t) &= C x(t) + D u(t),\end{aligned}\tag{22}$$

2.1.1 Rotary pendulum

Question 1. Based on the output and state defined above in Equations (11) and (12), derive the linear state-space model of the pendulum system, e.g. find the state-space matrices A , B , C and D in Equation (22). Why are there only two variables defined in the output equation?

Note: The input is the applied motor voltage $u(t) = v_m(t)$.

Question 2. Write a script that models the rotary pendulum using the state-space representation matrices derived from previous question.

The code for this Question is located at:

Pendulum\0_interfaceing_identification_modeling\02_state_space_modeling

Use the corresponding code in `digital_twin` for the simulation and `hardware` for the physical system.

- For the digital twin (simulation), go to `digital_twin`. In this folder, use the model parameters defined in the `qs3_rotpen_param.m` script supplied and the `qs3_rotpen_ABCD_eqns_down_student.m` as a starting point.

After completing the matrices, run

`qs3_setup_rotpen_ss_model.m`

in the MATLAB Workspace to generate a state-space object.

- For the hardware setup, go to `hardware`. In this folder, use the model parameters defined in the `qube2_rotpen_param.m` script supplied and the `qs2_rotpen_ABCD_eqns_down_student.m` as a starting point.

After completing the matrices, run

`qs2_setup_rotpen_ss_model.m`

in the MATLAB Workspace to generate a state-space object.

The A and B matrices should be displayed in the Command Window.

A =

0	0	1	0
0	0	0	1

0	152.01	-12.254	0.50048
0	-264.31	12.112	-0.87024

B =

0
0
50.637
-50.048

Question 3. Verify your model for the non-inverted pendulum case with the steps and questions below

1. Open the Simulink model

qs3_pen_ss_model.slx (For digital twin)

qs2_pen_ss_model.slx (For real hardware)

whose contents is shown in Figure 7. It applies for 20s a 0 – 1V; 1 Hz square wave to the pendulum system and to the physics-based state-space model.

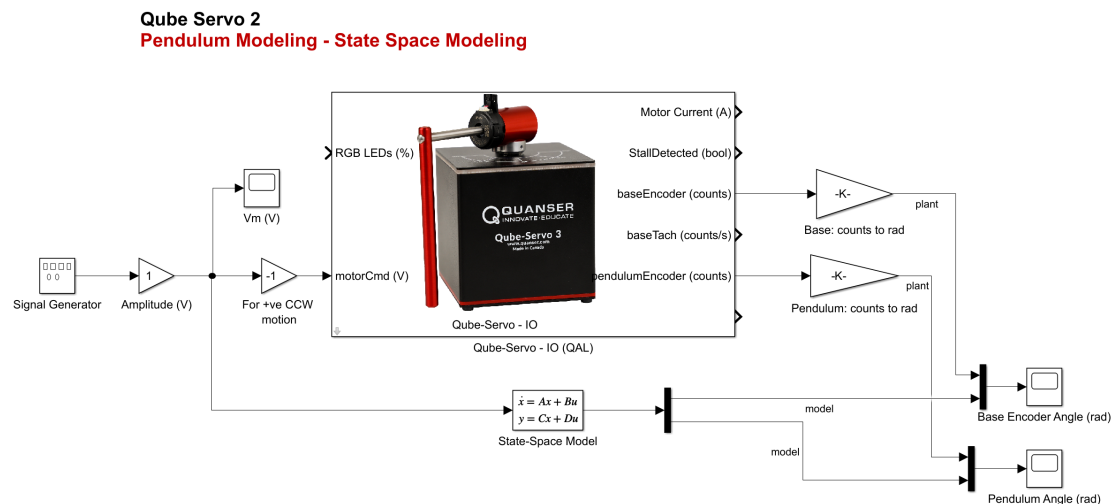


Figure 7: Simulink model comparing the response of the Qube-Servo and the linear state-space model

2. Place manually the pendulum close to the 0° position. Build and run the QUARC controller. Capture a screenshot of the Base Encoder Angle (rad) scope and the Pendulum Angle (rad) scope. The response should look similar to Figure 8.

3. In the Matlab scripts

qs3_rotpen_param.m (For digital twin)

qube2_rotpen_param.m (For real hardware)

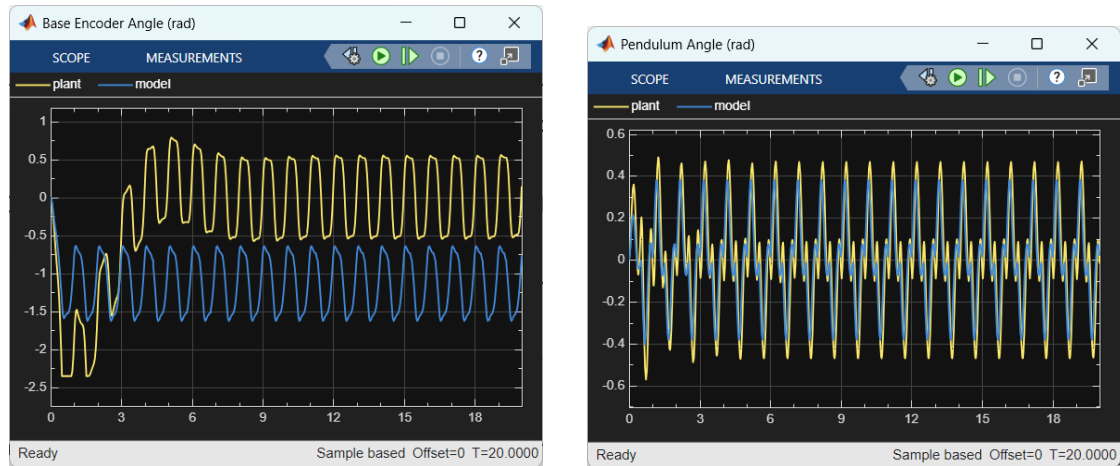


Figure 8: Sample response of the measured vs. modeled pendulum system

, the rotary arm viscous damping coefficient D_p and the pendulum viscous damping coefficient D_y were found experimentally to reasonably reflect the viscous damping of the system due to friction during a step response. However, those can vary slightly from system to system. If the model does not accurately represent the measured response, try modifying the damping coefficients **br** and **bp** to obtain a more accurate response.

4. Does your model represent the actual pendulum-down well? Give possible reasons why they wouldn't match.

The code used for following Questions 4 and 8 is located at:

Pendulum\1_control\11_BalanceControl_PolePlacement

Use the corresponding code in **digital_twin** for the simulation and **hardware** for the physical system. Use the script **qs3_setup_rotpen_ss_model.m** as entry point for writing code.

Question 4. After validation of the non-inverted pendulum model, analyse the system by answering following questions:

1. Determine open-loop poles of the inverted pendulum($\alpha \sim \pi$). Is the system stable?
2. Evaluate the controllability and observability of the rotary inverted pendulum system.

2.1.2 2-DOF helicopter

Question 5. Similar to Question 1, derive state-space matrices from given linear model in (13) and (14).

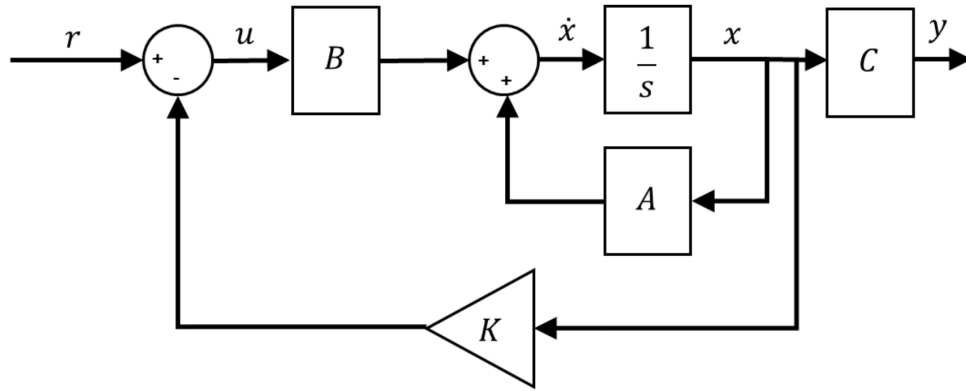


Figure 9: State feedback block diagram

Question 6. Write a script that models the 2-DOF helicopter using the state-space representation matrices derived from previous question. The code for this Question is located at:

```
2DOF_Helicopter\0_interfacing_identification_modeling
\02_state_space_modeling\digital_twin
```

Question 7. Similar to Question 4.

2.2 Design and implement of PD controller using pole placement method

The state-feedback control of a system is shown in Figure 9. Let u be a state feedback control law of the form

$$u = -Kx \quad (23)$$

where K is the feedback gain. Applying this to Equation 22 gives the closed-loop system:

$$\dot{x} = Ax - BKx = (A - BK)x \quad (24)$$

If (A, B) is controllable, we can use the state-feedback to place the poles at desired locations, e.g. in the left half of the s-plane. Thus we can find a gain K for the closed-loop system in Equation (24) to obtain a desired characteristic equation:

$$\prod_{i=1}^n (s - \lambda_i) = s^n + a_{n-1}s^{n-1} + \dots + a_1s + a_0 = 0 \quad (25)$$

which is equivalent with $\det |sI - (A - BK)| = 0$. We can use the MATLAB command `place` to generate the matrix gain K .

```
P = [lambda_1, ..., lambda_n]
K = place(A,B,P)
```

Attention: Pole placement can be ill-conditioned if unrealistic pole locations are chosen. It is important to first test the model in simulation and verify both its performance and validity before deploying it to the digital twin or the real system.

2.2.1 Rotary pendulum

Question 8. From the inverted pendulum model obtained above, design PD controller using pole placement method. The time-domain specifications when the pendulum is balanced and tracking a rotary arm setpoint are:

$$\begin{aligned} PO &\leq 6.81\% \\ t_s &\leq 1.54 \text{ s} \end{aligned} \tag{26}$$

1. The percent overshoot and settling time specifications given translates into the following natural frequency and damping ratio requirements:

$$\begin{aligned} \zeta &= 0.65 \\ \omega_n &= 4 \text{ rad/s} \end{aligned}$$

Based on these specifications, find the location of the two dominant poles p_1 and p_2 .

2. Find the desired characteristic equation if the other poles are placed at $p_3 = -40$ and $p_4 = -45$.
3. Write script in Matlab to find the state-feedback gain, K , required to place the closed-loop poles to the desired locations. Give the Matlab commands used and the obtained matrix gain K .

Question 9. Use following instructions to implement the designed feedback controller for balancing the inverted pendulum

1. Open simulink model
`qs3_bal_pp.slx` (For digital twin)
`qs2_bal_pp.slx` (For real hardware)
2. Set the variable K in the Matlab workspace.
3. Set first the Gain block that is connected to the Signal Generator to 0. Then, set the Signal Generator block to the following:
 - Type = Square
 - Amplitude = 1
 - Frequency = 0.125 Hz



Figure 10: Lift pendulum button in virtual environment

4. Click on **Monitor & Tune** to build and run the file. When the Qube-Servo turns green, gently and manually rotate the pendulum in the upright position until the controller engages. In virtual environment, use the button **Lift Pendulum** as shown in Figure 10.
5. Once the pendulum is balanced, set the Gain to a desired gain(e.g. 30) to change the arm angle.
6. Does the rotary arm and pendulum response satisfy the specification? If not, give one reason why there is a discrepancy. Modify the desired closed-loop poles to get better results.

2.2.2 2-DOF helicopter

Question 10. From the helicopter model obtained above, design PD controller using pole placement method satisfying the following specifications:

- Peak time: $t_p \leq 2.5 \text{ s}$ for pitch angle and $t_p \leq 3.5 \text{ s}$ for yaw angle.
- Percent Overshoot: $PO \leq 5\%$ for both axes.

Write script in Matlab to find the state-feedback gain, K , required to place the closed-loop poles to the desired locations. The code used for this section is located at

2DOF_Helicopter\1_control\11_Control_PolePlacement\digital_twin

Use `aero2_2dof_control_pp.m` as script for entry simulation.

Question 11. Use following instructions to implement the designed controller for the helicopter:

- The `s_aero2_2dof_pp.slx` simulink model shown in Figure 11 can be used to simulate the pitch and yaw control of the Aero 2 - 2-DOF Helicopter.
- The `q_aero2_2dof_pp.slx` simulink model shown in Figure 12 will be used with QUARC to run the PD control on the digital twin or the real hardware.

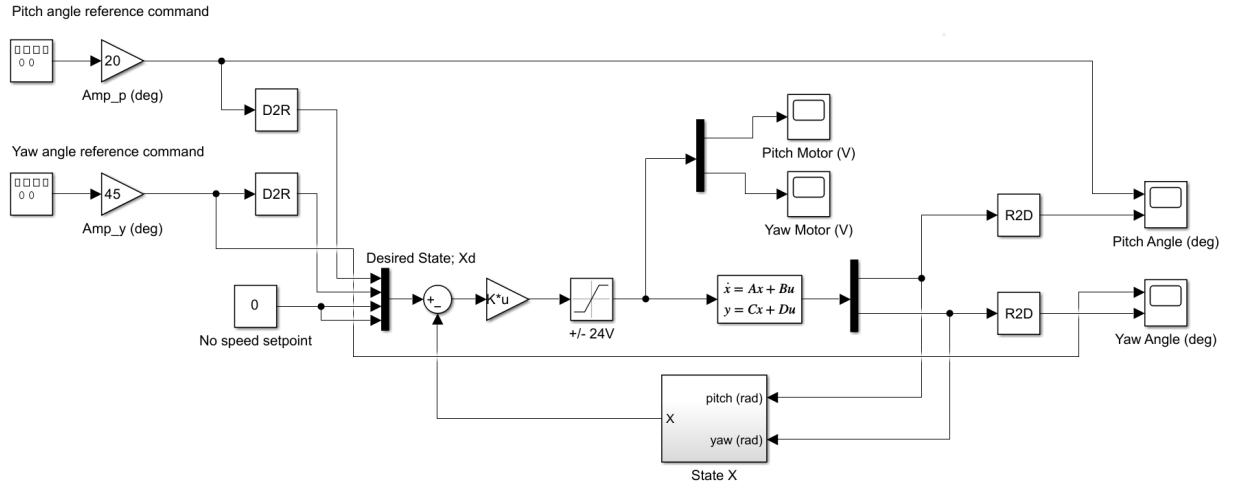


Figure 11: Simulates the control of the pitch and yaw axes on the 2 DOF Aero 2.

The Simulink model uses the **HIL Write Analog** and **HIL Read Timebase** blocks from the **QUARC Targets** library to apply the control input voltage to the rotors and measure the corresponding pitch and yaw response. The response is saved into a MATLAB ***.mat** files using the **ToHostFile** block.

- First, run `aero2_2dof_control_pp.m` to load the system matrices A , B , C , and D , and to define the designed state-feedback gain K . The simulation of the model will then be executed to evaluate the performance of the controller. You may further tune the parameters to meet the desired specifications. Record your tuning process and all corresponding feedback gains K along with the simulation results.
- Then, run the simulink model `q_aero2_2dof_pp.slx` to execute your designed controller on the digital twin model.
- Finally, run the script `aero2_2dof_pp_analyze.m` to plot and analyze the results of the virtual simulation.

2.3 Design and implement LQR controller

The Linear Quadratic Regulator (LQR) optimization method can be used to find the control gain. Given the state-space model of the system (22), the LQR algorithm computes a control law u to minimize the performance criterion or cost function:

$$J = \int_0^{\infty} \left(x(t)^{\top} Q x(t) + u(t)^{\top} R u(t) \right) dt, \quad (27)$$

The design matrices Q and R hold the penalties on the deviations of state variables from their setpoint and the control actions, respectively.

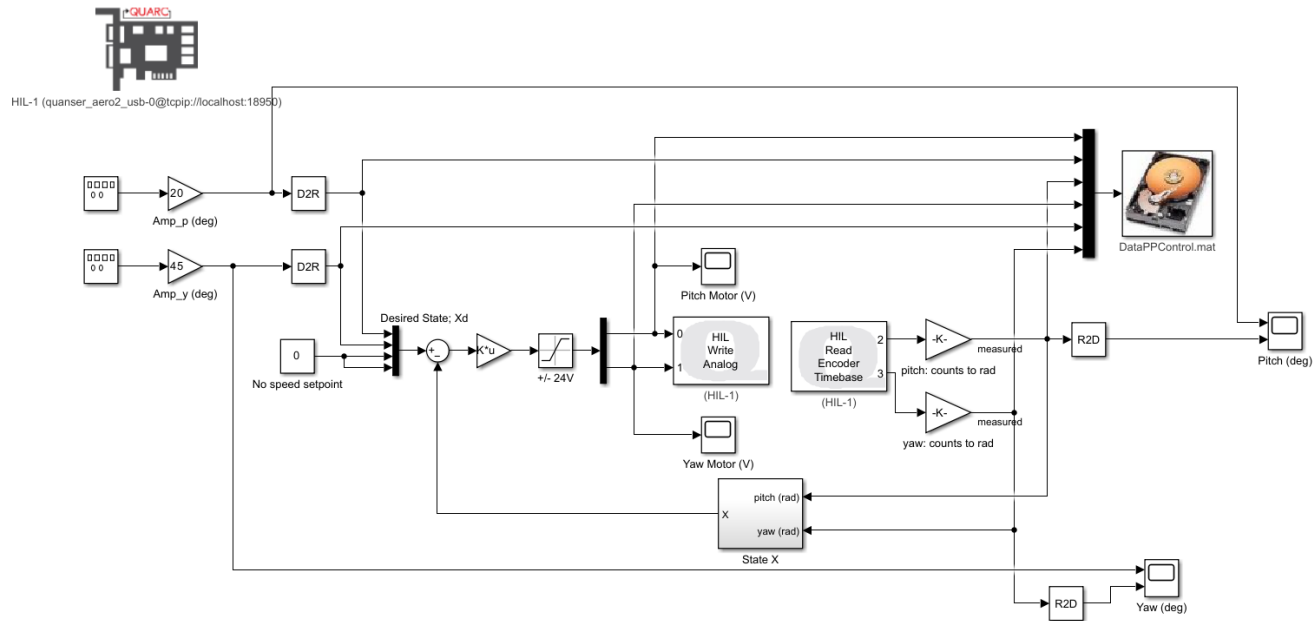


Figure 12: Simulink model used with QUARC to implement the controller on the Aero 2 system.

2.3.1 Rotary pendulum

The code used for the following Questions 12 and 13 is located at:

Pendulum\1_control\12_BalanceControl_LQR

Use the corresponding code in `digital_twin` for the simulation and `hardware` for the physical system. Use the script `qs_setup_rotpen_ss_model.m` as entry point for writing code.

Question 12. Design LQR controller so that the criteria below are met.

- Pendulum angle overshoot is less than $\pm 3^\circ$ ($\alpha < |3^\circ|$)
- Overshoot of the base angle is less than 14%.

Write the script in Matlab to find the feedback gain K . To find good tuning parameters, one can use the guides below:

1. Start with $R = 1$ and $Q = I_n$, calculate the control gain K . Take a screenshot of the control gain generated.
2. Calculate the poles of closed-loop system $A - BK$. All the poles now should be stable.
3. Repeat adjusting Q and R until satisfying the specifications.

Note: Record all the results obtained and make a comparative analysis for your tuning process. Describe your experimental procedure to find the necessary control gain.

Hint:

- Explore the effects on the control gain, K , of changing each of the diagonal elements in Q to 5, while keeping all other diagonal elements at 1.
- Write down the effects of changing each of the diagonal elements in the weighting matrix Q .

Question 13. Follow similar steps in Question 9. Replace the simulink with model `qs_bal_lqr.slx`. At the end, make a comparison, with the PD control above.

Take a screenshot of scopes demonstrating the new behavior. The measurement tool can be used to measure the performance of the controller as shown in Figure 13 below:

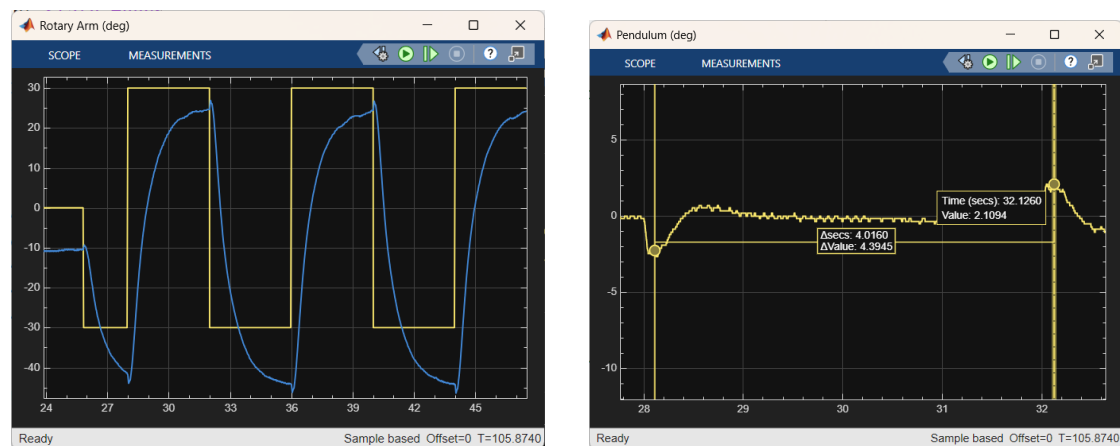


Figure 13: Sample response of an LQR controller

2.3.2 2-DOF helicopter

Question 14. With the same specifications given in 10, design and write Matlab script to find the gain of feedback law.

Use the code in

`2DOF_Helicopter\1_control\12_Control_LQR\digital_twin`

for this Question. The simulink models

`s_aero2_2dof_lqr.slx`

`q_aero2_2dof_lqr.slx`

works as similar as in Pole placement section.

Use the script `aero2_2dof_control_lqr.m` for designing the controller and testing in simulation. Otherwise, use `aero2_2dof_lqr_analyze.m` to analyze the test run on the digital twin system.

Question 15. Implement the Quanser Aero system by following similar instructions given in Question 11. Make a comparative analysis between LQR and PD control design for the Aero helicopter system.

Question.* Analyze for both systems their behaviors under input saturation bounds (check the input/output constraints that the dynamical model(22) needs to respect). Which controller that we used above can handle these constraints?